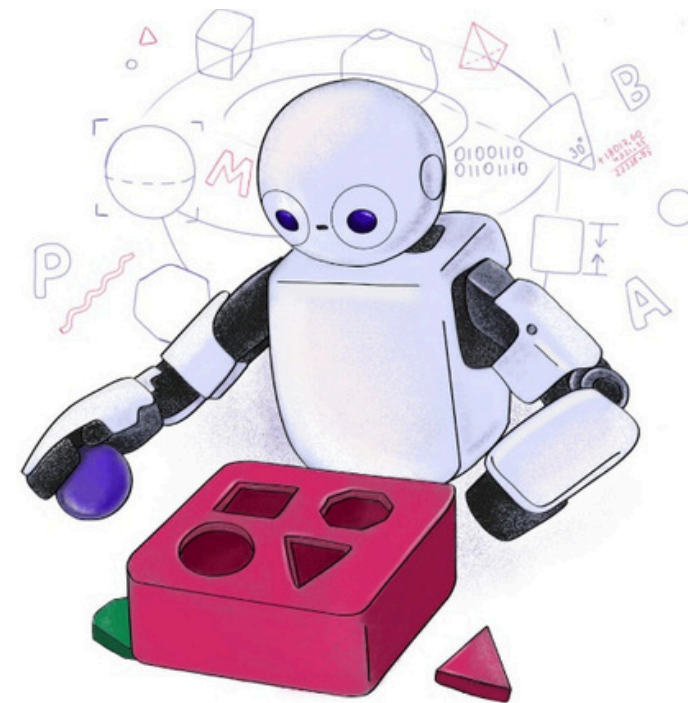


TP558 - Tópicos avançados em Machine Learning: ***DINOv3: Self-supervised learning***



Inatel

Rafaella Laureano Dias
rafaella.dias@mtel.inatel.br

Introdução

O aprendizado autossupervisionado elimina a necessidade de anotações manuais, permitindo escalar modelos para grandes conjuntos de dados. O DINOv3 representa um avanço nesse paradigma ao introduzir estratégias simples e eficazes que produzem representações visuais densas e versáteis, superando o estado da arte em diversas tarefas de visão computacional.



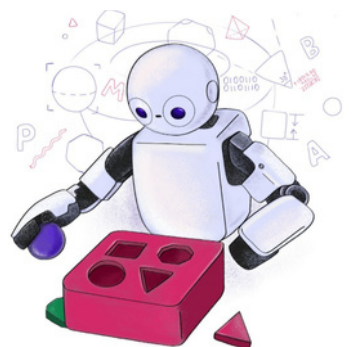
DINO



DINOv2



DINOv3



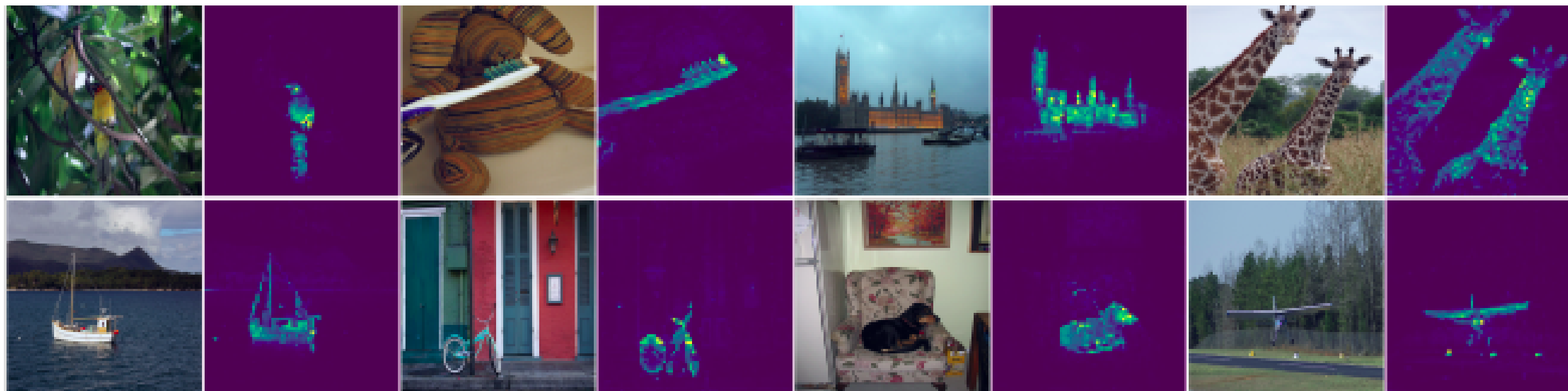
Fundamentação teórica

Aprendizado autosupervisionado (Componentes Básicos):

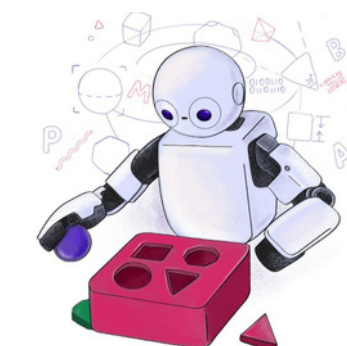
Encoder (Codificador): É uma rede neural que recebe a entrada bruta (imagem, texto, áudio) e transforma em uma representação interna (vetor ou embedding) que captura as características mais relevantes do dado. É normalmente uma rede neural profunda, como ResNet ou um ViT.

Pretext Task (Tarefa de Pretexto): O modelo é treinado em uma tarefa artificial criada a partir dos próprios dados. Por exemplo:

- Em visão computacional, o encoder pode receber uma versão da imagem com partes removidas ou distorcidas, e o modelo precisa reconstruir a imagem original (autoencoder) ou prever a transformação aplicada (rotacionar a imagem).
- Em NLP, o modelo pode tentar prever palavras faltantes em uma frase.
- Essa tarefa gera supervisão implícita, orientando o modelo a aprender representações significativas.



A figura apresenta imagem com uma autoatenção saindo do token CLS, que é um token aprendível que é adicionado manualmente como a entrada. (ViT patches 8x8)



Fundamentação teórica



Aprendizado autosupervisionado (Componentes Básicos):

Projection Head (Cabeça de Projeção): Em muitos modelos, após o encoder, há uma camada adicional que projeta a representação codificada para um espaço onde a tarefa de aprendizado é otimizada, por exemplo, o encoder gera um vetor com 768 dimensões, o projection head transforma esse vetor em outro espaço (ex.: 128 dimensões) onde faz sentido aplicar uma loss contrastiva ou de distilação. Geralmente usa-se MLP (Multi-Layer Perceptron) de 2-3 camadas.

Loss Function (Função de Perda): A função de perda é definida para que o modelo aprenda a minimizar a diferença entre as previsões feitas a partir da representação interna e a "verdade" derivada da tarefa de pretexto (por exemplo, erro na reconstrução da imagem, distância entre embeddings, ou previsões corretas das partes ocultas do dado). As funções de perda podem ser:

- Contrastive loss (empurrar embeddings de imagens diferentes e aproximar embeddings da mesma imagem).
- Cross-entropy entre student e teacher (DINO “distillation with no labels”).
- Reconstruction loss (autoencoders).

1. Encoder → extrai as features.
2. Projection head → transforma essas features para um espaço de treino.
3. Pretext task → define como gerar os dados de treino (ex.: duas views da mesma imagem).
4. Loss function → calcula o erro e atualiza os pesos do encoder + head.

No fim, quando o treinamento acaba, geralmente descarta o projection head e usa só o encoder como extrator de características para tarefas (classificação, detecção, segmentação etc.).

Arquitetura e funcionamento

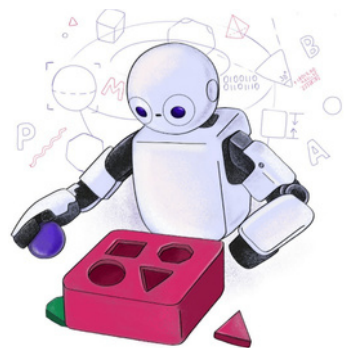
O DINOv3, é uma recente evolução da família de modelos de aprendizado autossupervisionado, realizando grandes melhorias em relação ao DINOv2.

1.Escala de dados (mais dados e melhor mistura).

Coleta de **17 bilhões** de imagens de posts públicos do Instagram (já moderados). A partir desse pool bruto, o paper cria três partes de dados:

- Usa o DINOv2 para clusterizar em 5 níveis: 200M → 8M → 800k → 100k → 25k clusters. Clusterizar = agrupar dados parecidos em clusters (grupos).
- Aplicam um balanceador, que seleciona imagens de forma equilibrada entre os clusters, garantindo cobertura uniforme de conceitos. O resultado é o dataset LVD-1689M com 1,689 milhão de imagens curadas
- Adiciona dataset públicos (ImageNet-1k/22k, Mapillary). Durante o pré-treino, o sampler mistura as 3 partes (heterogêneo) e, em 10% das iterações, usa um lote homogêneo só de ImageNet-1k.

Teacher model	DINOv2	DINOv3
Backbone	ViT-giant	ViT-7B
#Params	1.1B	6.7B
#Blocks	40	40
Patch Size	14	16
Pos. Embeddings	Learnable	RoPE
Registers	4	4
Embed. Dim.	1536	4096
FFN Type	SwiGLU	SwiGLU
FFN Hidden Dim.	4096	8192
Attn. Heads	24	32
Attn. Heads Dim.	64	128
DINO Head MLP	4096-4096-256	8192-8192-512
DINO Prototypes	128k	256k
iBOT Head MLP	4096-4096-256	8192-8192-384
iBOT Prototypes	128k	96k



Arquitetura e funcionamento

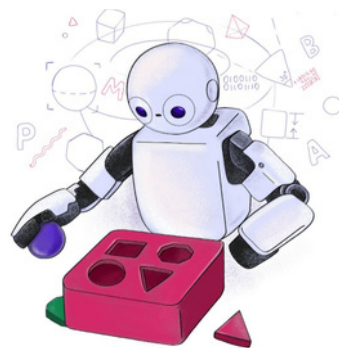
2.Professor (teacher) e backbone

Teacher do DINOv3: ViT-7B (6.7B parâmetros), patch 16 (vs 14), RoPE como positional embedding, embedding 4096-d, 32 cabeças, etc. (mantém 40 blocos).

*RoPE (Rotary Positional Embeddings): forma relativa de posicionamento (aplica rotações dependentes da posição nos pares Q/K da atenção), melhorando extrapolação e robustez a mudanças de resolução/escala. MLP maior.

Antes de prosseguir, o que é teacher e backbone?

Teacher model	DINOv2	DINOv3
Backbone	ViT-giant	ViT-7B
#Params	1.1B	6.7B
#Blocks	40	40
Patch Size	14	16
Pos. Embeddings	Learnable	RoPE
Registers	4	4
Embed. Dim.	1536	4096
FFN Type	SwiGLU	SwiGLU
FFN Hidden Dim.	4096	8192
Attn. Heads	24	32
Attn. Heads Dim.	64	128
DINO Head MLP	4096-4096-256	8192-8192-512
DINO Prototypes	128k	256k
iBOT Head MLP	4096-4096-256	8192-8192-384
iBOT Prototypes	128k	96k

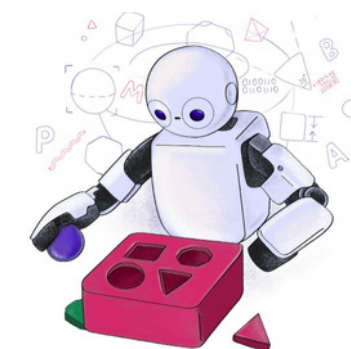
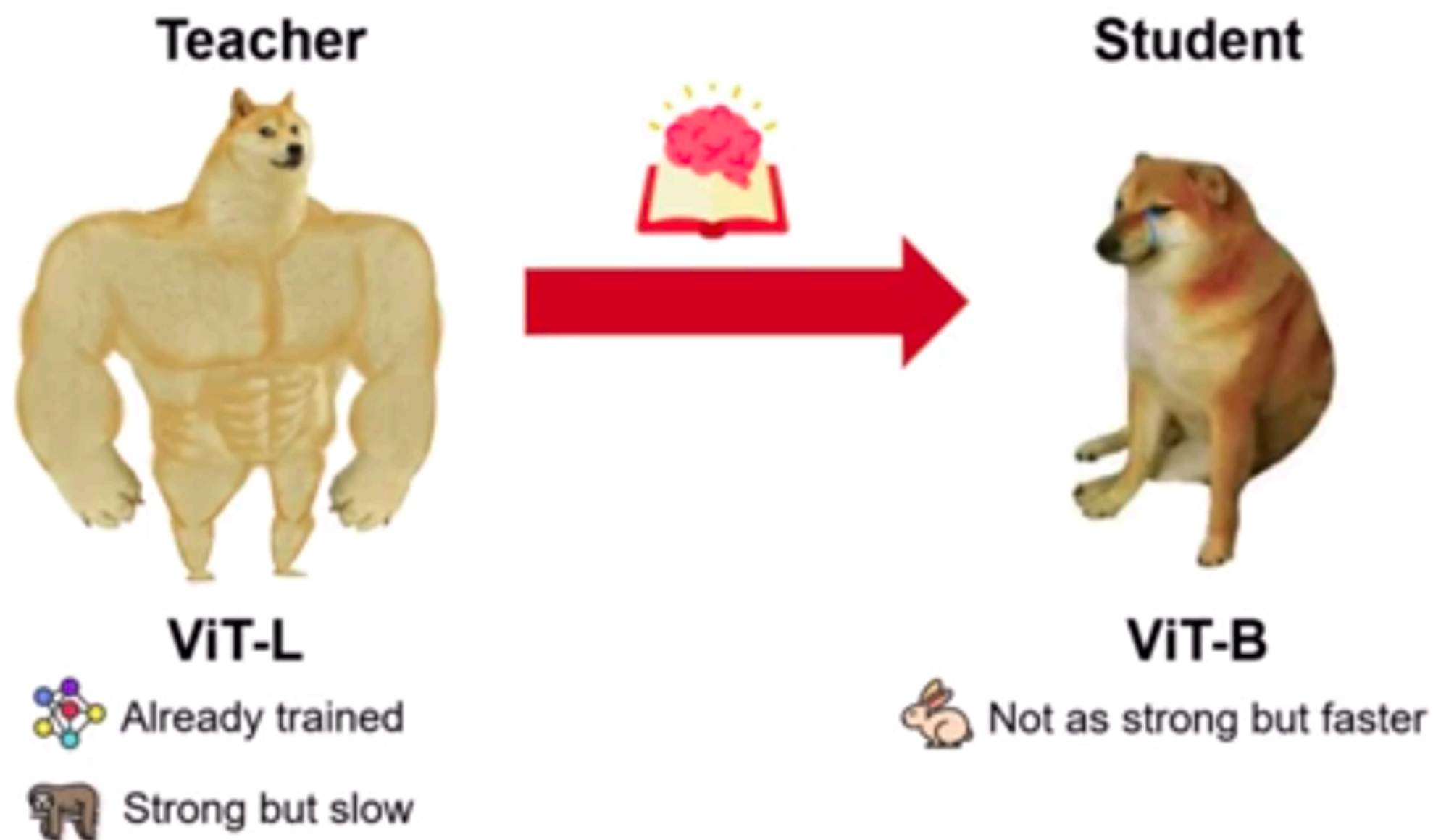


Arquitetura e funcionamento

Backbone: é o “cérebro” que olha a imagem e gera os vetores de representação.

Teacher: é como o “professor mais estável”, que já tem conhecimento e guia o “aluno” (student) a aprender, mesmo sem labels.

- O teacher recebe uma versão da imagem (com certos augmentations).
- O student recebe outra versão diferente da mesma imagem.
- Ambos produzem vetores (ou distribuições).
- A loss function compara essas duas saídas e força o student a imitar o teacher.



Arquitetura e funcionamento

3. Combinação de 3 perdas (objetivos do pré-treino) e otimização estável (sem schedulers)

$$L_{pre} = L_{dino} + L_{iBOT} + 0.1 * L_{DKoleo}$$

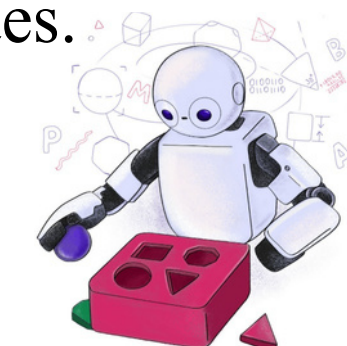
- L_{dino} (global/instância): aproxima crops globais da mesma imagem e distancia imagens diferentes (via teacher EMA).
- L_{iBOT} (local/patch): mascara uma fração de tokens de patch do aluno e os alinha com tokens visíveis do professor (patch-wise).
- L_{DKoleo} : regulariza a dispersão das features (evita colapso), aplicada a mini-lotes de 16 amostras, possivelmente distribuídos entre GPUs.

EMA (Exponential Moving Average - Média Móvel Exponencial), é uma forma de atualizar os pesos do teacher a partir dos pesos do student, de maneira suave e estável.

Sem schedulers (cronograma de hiperparâmetros): learning rate, weight decay e EMA do teacher constantes.

Benefícios:

- (i) pode-se continuar treinando enquanto métricas downstream melhorarem;
- (ii) menos hiperparâmetro



Arquitetura e funcionamento

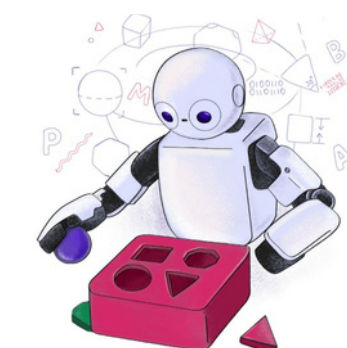
4. Gram Anchoring

Quando o treino é estendido, a acurácia global (ex.: ImageNet linear) continua subindo, mas o desempenho em tarefas densas (ex.: segmentação, mIoU) cai após ~200k iterações. Isso ocorre porque os patch tokens tornam-se cada vez mais parecidos com o CLS token (representação global). Resultado: os mapas de similaridade entre patches ficam difusos/ruidosos, prejudicando a granularidade espacial necessária para segmentação e depth (profundidade).

Para recuperar a consistência local, o DINOv3 não força cada patch do aluno a copiar o do professor. Em vez disso, atua sobre a matriz de Gram (similaridade entre patches). O objetivo é que o aluno preserve as relações de similaridade entre patches (quem é parecido com quem), tomando como referência um Gram teacher (snapshot do teacher EMA de uma fase inicial, quando as features locais ainda eram nítidas).

Então aproxima $X_S X_S^T$ de $X_G X_G^T$ com erro quadrático (norma de Frobenius). As features podem se mover no espaço, contanto que a geometria de similaridades fique boa. É isso que salva segmentação/depth no treino longo.

$$L_{\text{Gram}} = \left\| X_S X_S^T - X_G X_G^T \right\|_F^2$$

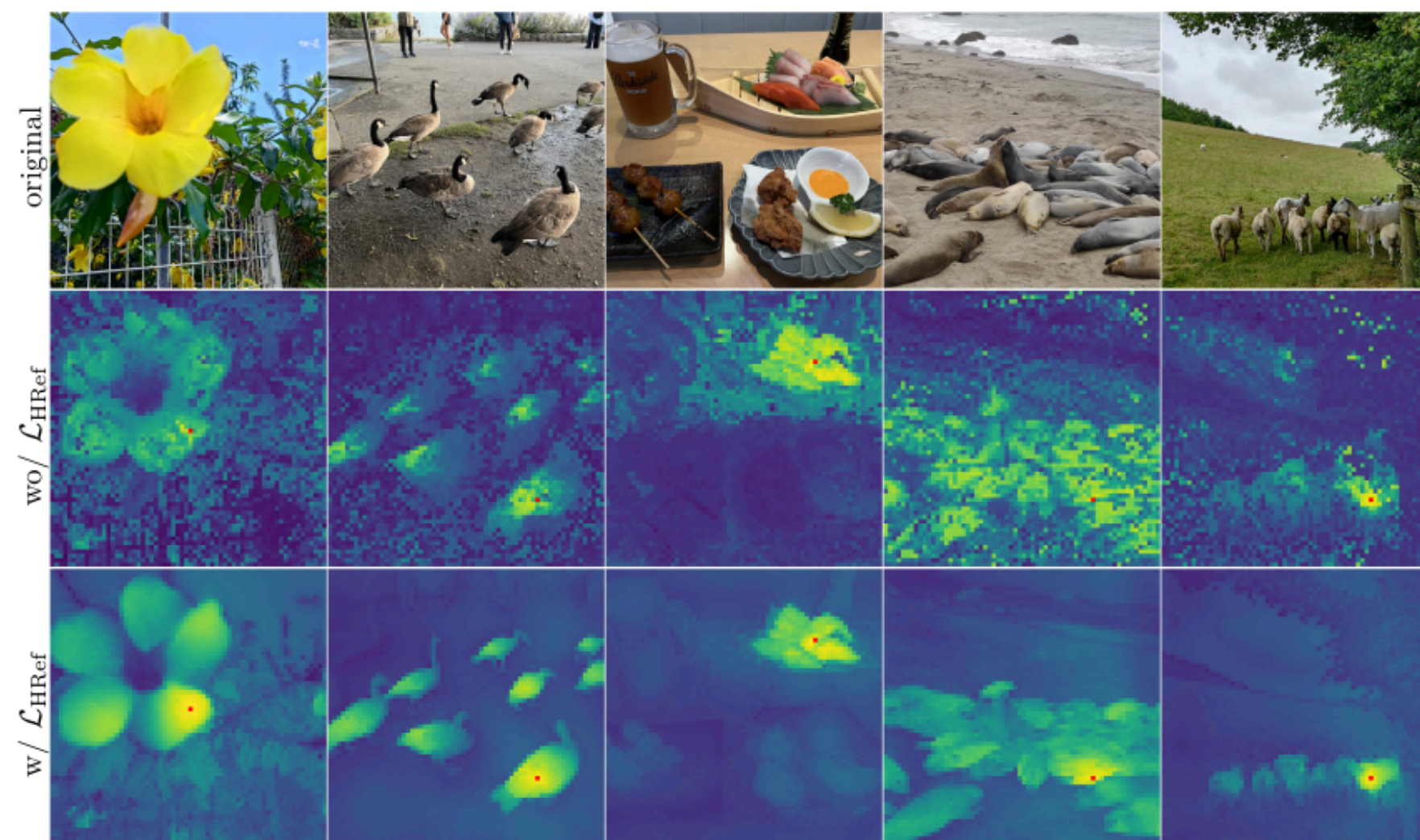


Arquitetura e funcionamento

Pós-treinamento: adaptação à alta resolução

O backbone é treinado em 256 px, mas aplicações pedem 512–4096 px. Para isso, aplicam um pós-treinamento curto (10k iterações) com crops mistos de maior resolução. O **Gram Anchoring** continua essencial nessa fase, sem ele, as tarefas densas degradam. Ou seja:

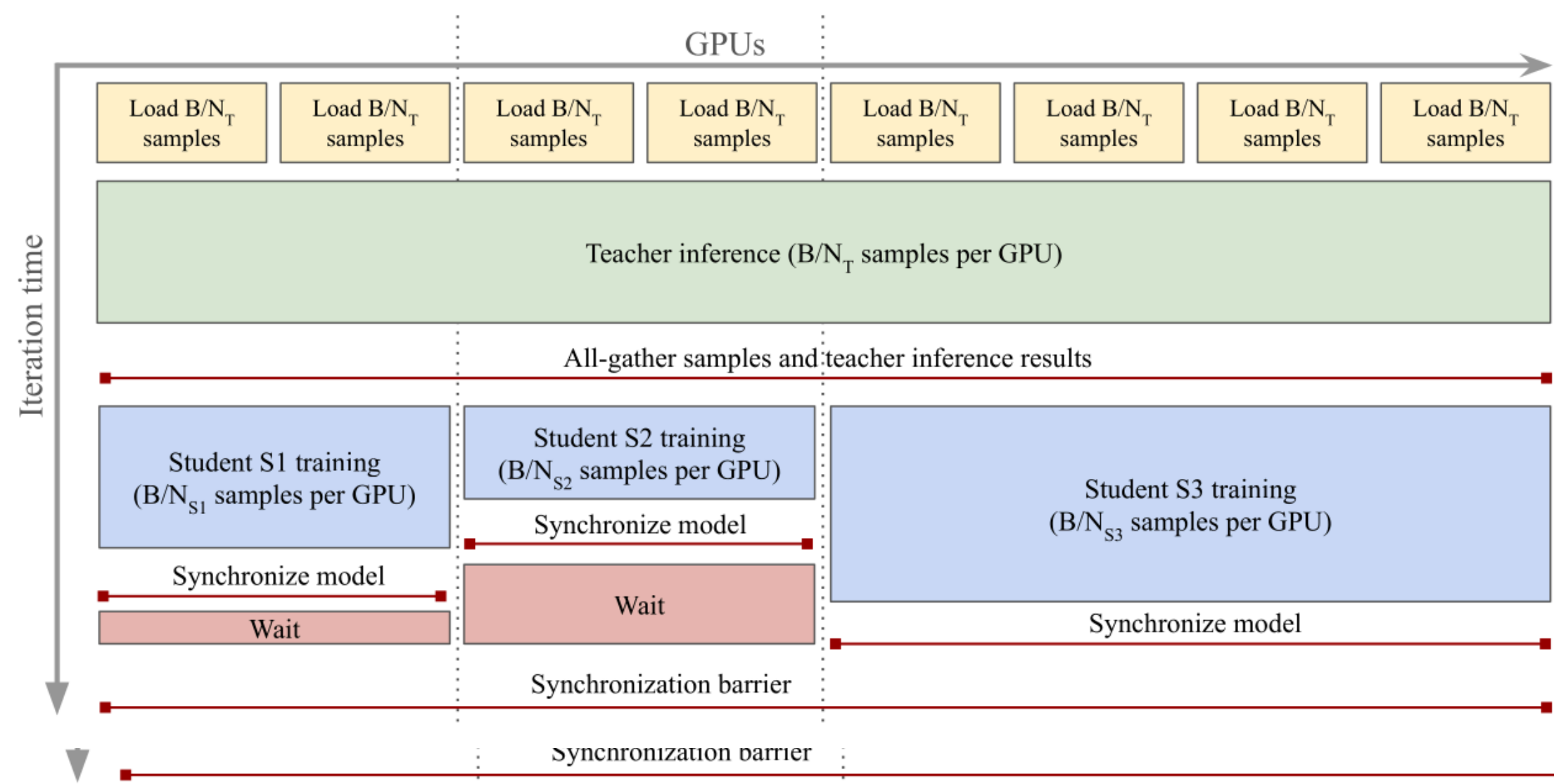
- Treinar direto em 512–4096 px seria caro demais.
- Então eles fazem só um ajuste final curto, misturando resoluções.
- O Gram anchoring garante que os detalhes locais não se percam nessa transição.



Arquitetura e funcionamento

Distillation multi-aluno

O custo principal é a inferência do professor 7B. Para eficiência, o artigo introduz destilação multi-aluno: vários students (ViT-S, B, L, H+) são treinados em paralelo, compartilhando a inferência única do professor por iteração. Assim, o conhecimento do modelo gigante é transferido para uma família de modelos menores e práticos.



- Parte amarela (topo): carregamento dos lotes de dados em cada GPU (B/N_T amostras).
- Parte verde: inferência do professor feita em paralelo, só uma vez por iteração.
- Linha vermelha: todos os students recebem os resultados do teacher (all-gather).
- Blocos azuis: treinamento dos students (S1, S2, S3) em paralelo, cada um com seu lote.
- Blocos vermelhos ("Wait"): mostram que há sincronização entre os alunos, e cada um atualiza seu modelo separadamente.
- Barreira de sincronização (fim da iteração): todos os students precisam alinhar antes de passar para o próximo passo.

Arquitetura e funcionamento



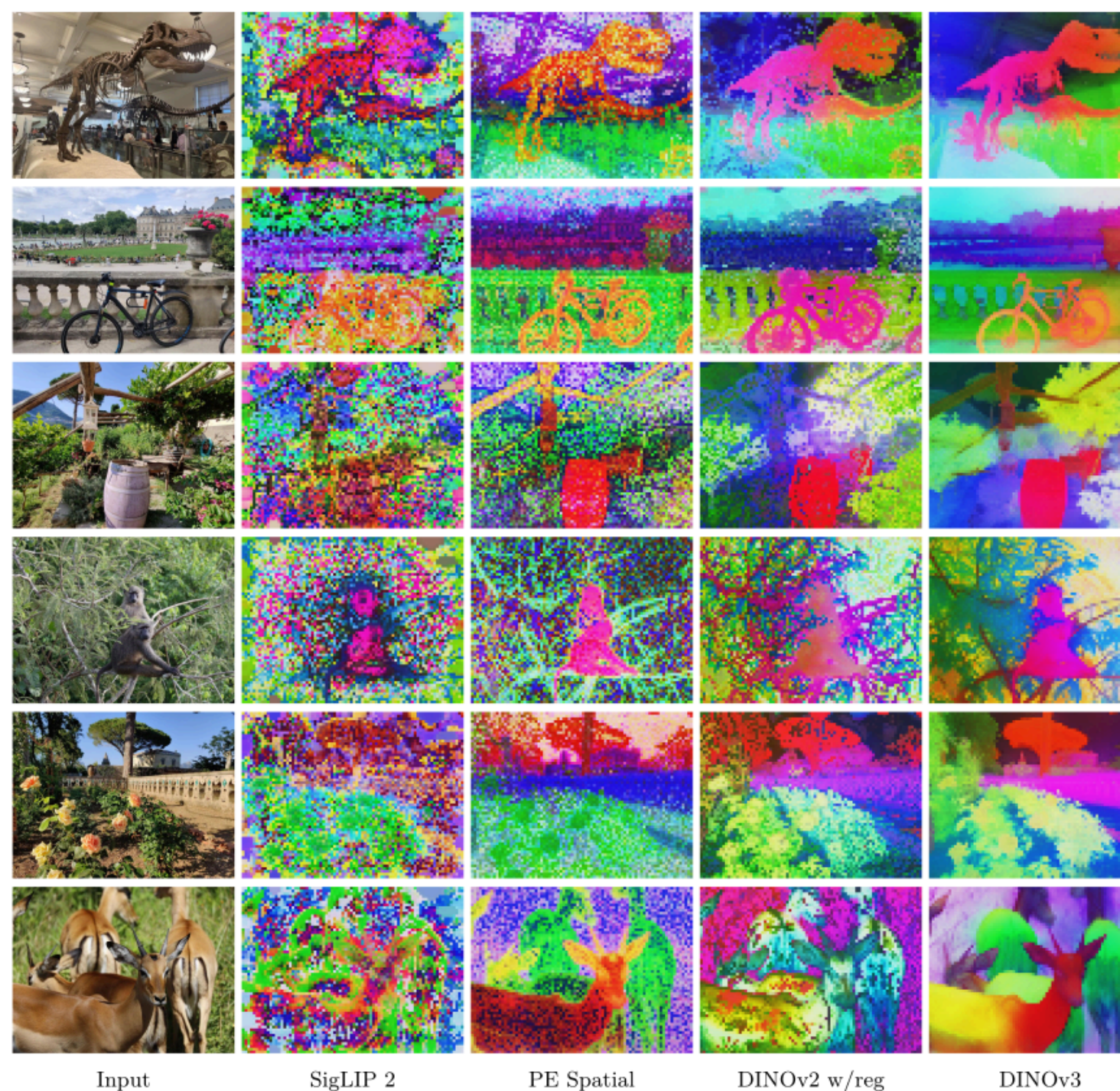
Resultados: o que o DINOv3 entrega

O DINOv3 7B/16 estabelece novo estado da arte em segmentação e estimativa de profundidade: atinge mIoU (mean Intersection over Union) mais alto em ADE20k (55.9), Cityscapes (81.1) e VOC (86.6), e reduz significativamente o RMSE (Root Mean Squared Error) em datasets de depth (NYUv2: 0.309; KITTI: 2.346), superando DINOv2 e demais métodos.

Method	ViT	Segmentation			Depth	
		ADE20k	Citysc.	VOC	NYUv2 ↓	KITTI ↓
<i>Agglomerative backbones</i>						
AM-RADIOv2.5	g/14	53.0	78.4	85.4	0.340	2.918
PEspatial	G/14	49.3	73.2	82.7	0.362	3.082
<i>Weakly-supervised backbones</i>						
SigLIP 2	g/16	42.7	64.8	72.7	0.494	3.273
PEcore	G/14	38.9	61.1	69.2	0.590	4.119
<i>Self-supervised backbones</i>						
Franca	g/14	46.3	68.7	82.9	0.445	3.140
DINOv2	g/14	49.5	75.6	83.1	0.372	2.624
Web-DINO	7B/14	42.7	68.3	76.1	0.466	3.158
DINOv3	7B/16	55.9	81.1	86.6	0.309	2.346

Arquitetura e funcionamento

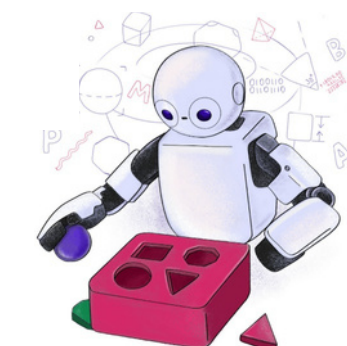
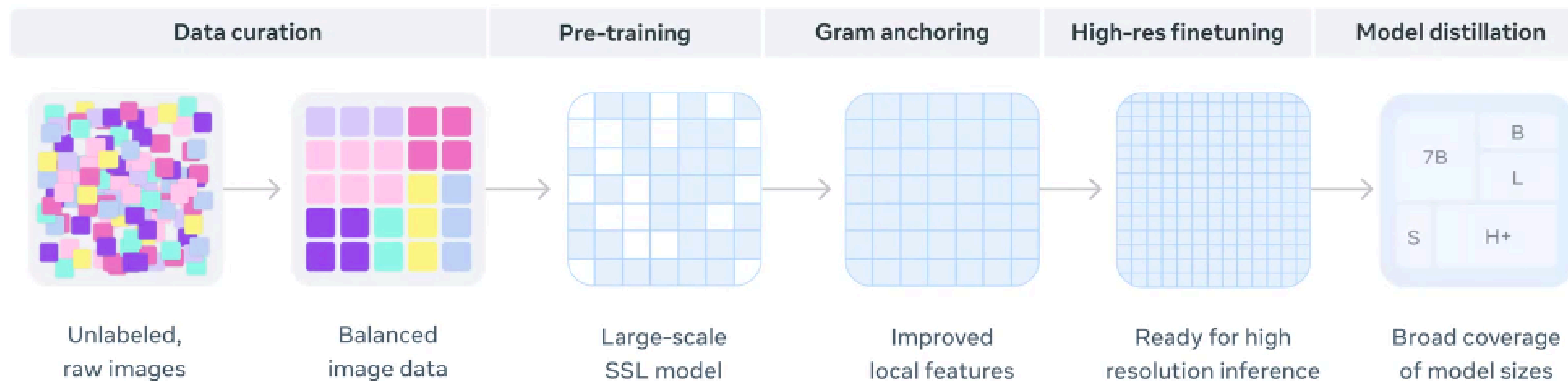
Qualidade das features densas (visual) (PCA→RGB): comparando SigLIP-2, PEspatial, DINOv2 (com registers) e DINOv3 7B/16, os mapas do DINOv3 são mais nítidos, com menos ruído e maior coerência semântica (mesma grid de 80×60 tokens).



Arquitetura e funcionamento

Possíveis limitações

- Custo alto: treinar 7B/1M+ iterações ainda é datacenter-level.
- Curadoria de dados: pipeline complexa (clustering + retrieval + balanceamento) é obrigatória.
- Gram teacher: precisa ser escolhido cedo (100–200k iterações) para evitar degradação.



Exemplos de aplicação



Os modelos pré-treinados estão disponíveis no github <https://github.com/facebookresearch/dinov3>

- PyTorch Hub (`torch.hub.load`)
- Hugging Face Hub (AutoModel / pipeline)
- Com pesos para diferentes tamanhos de ViT (S, B, L, H, 7B) e ConvNeXt, além de versões para imagens web (LVD-1689M) e satélite (SAT-493M)

Para um exemplo de aplicação foi utilizado um dataset disponível na plataforma Kaggle com imagens médicas chamado “Brain MRI segmentation” (<https://www.kaggle.com/datasets/mateuszbuda/lgg-mri-segmentation>)

Para acessar os pesos treinados é preciso requisitar para a meta e os tokens expiram diariamente.

Pretrained models

i Please follow the link provided below to get access to all the model weights: once accepted, an e-mail will be sent with the complete list of URLs pointing to all the available model weights (both backbones and adapters). These URLs can then be used to either:

- download the model or adapter weights to a local filesystem and point `torch.hub.load()` to these local weights via the `weights` or `backbone_weights` parameters, or
- directly invoke `torch.hub.load()` to download and load a backbone or an adapter from its URL via also the `weights` or `backbone_weights` parameters.

See the example code snippets below.

! Please use `wget` instead of a web browser to download the weights.

ViT models pretrained on web dataset (LVD-1689M):

Model	Parameters	Pretraining Dataset	Download
ViT-S/16 distilled	21M	LVD-1689M	[link]
ViT-S+/16 distilled	29M	LVD-1689M	[link]
ViT-B/16 distilled	86M	LVD-1689M	[link]
ViT-L/16 distilled	300M	LVD-1689M	[link]
ViT-H+/16 distilled	840M	LVD-1689M	[link]

Model card Files and versions xet Community 12

Edit model card

You need to agree to share your contact information to access this model

The information you provide will be collected, stored, processed and shared in accordance with the [Meta Privacy Policy](#).

Your request to access this repository has been submitted and is awaiting a review from the repository authors. You can check the status of all your access requests in [your settings](#).

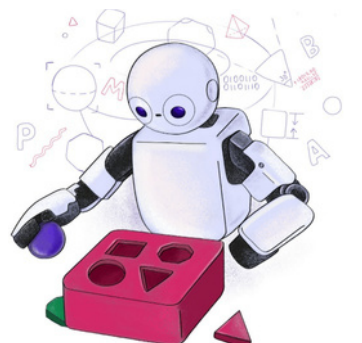
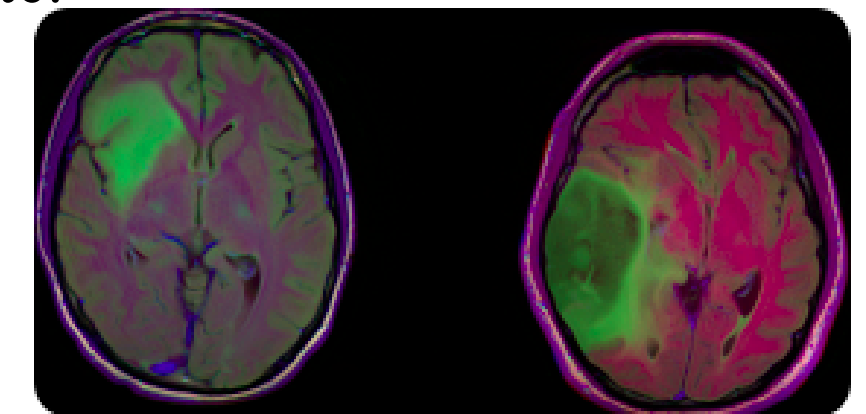
If approved, you'll get access to all 12 repositories in the [DINOv3](#) Gating Group Collection, including this one.

Confirmation

DINOv3 download request received

Thank you for submitting a request. We will email you with more information regarding your download.

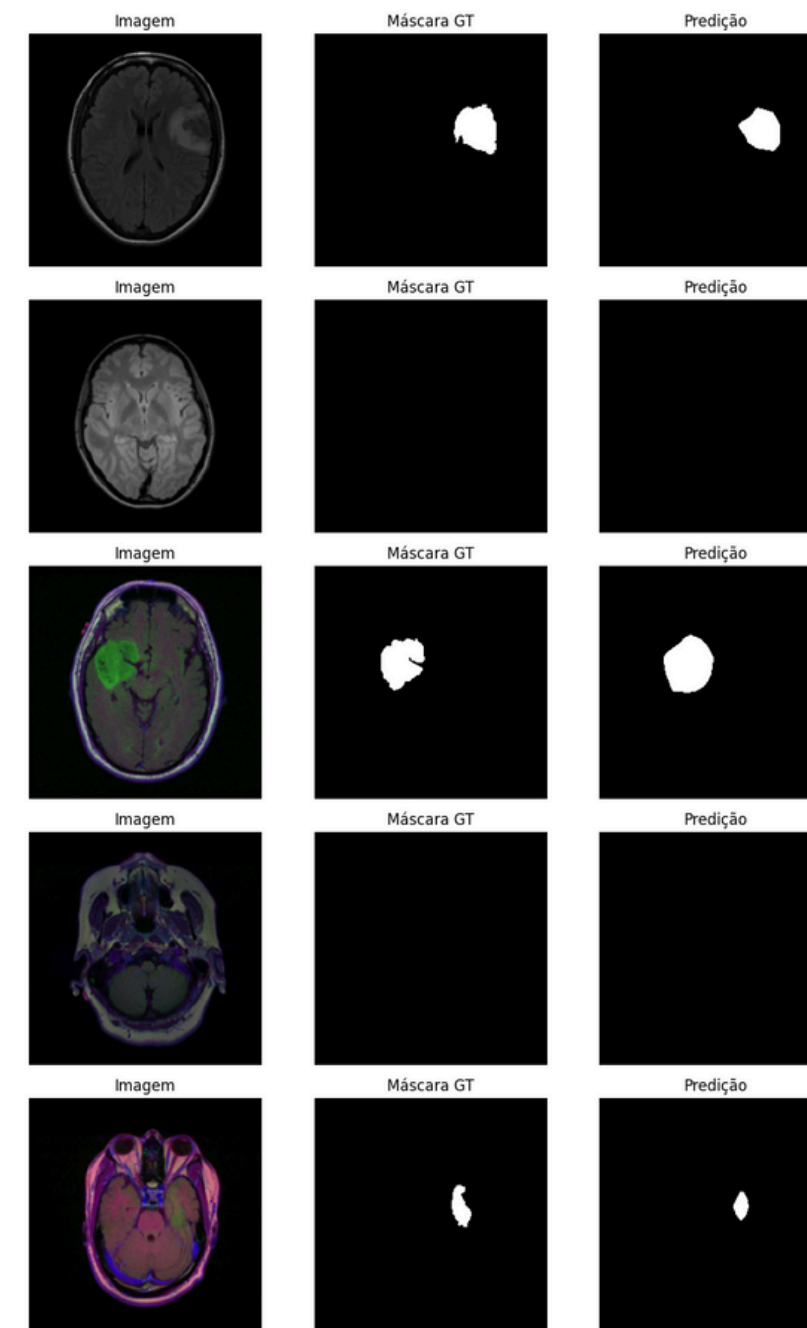
[Return to Home](#)



Exemplos de aplicação

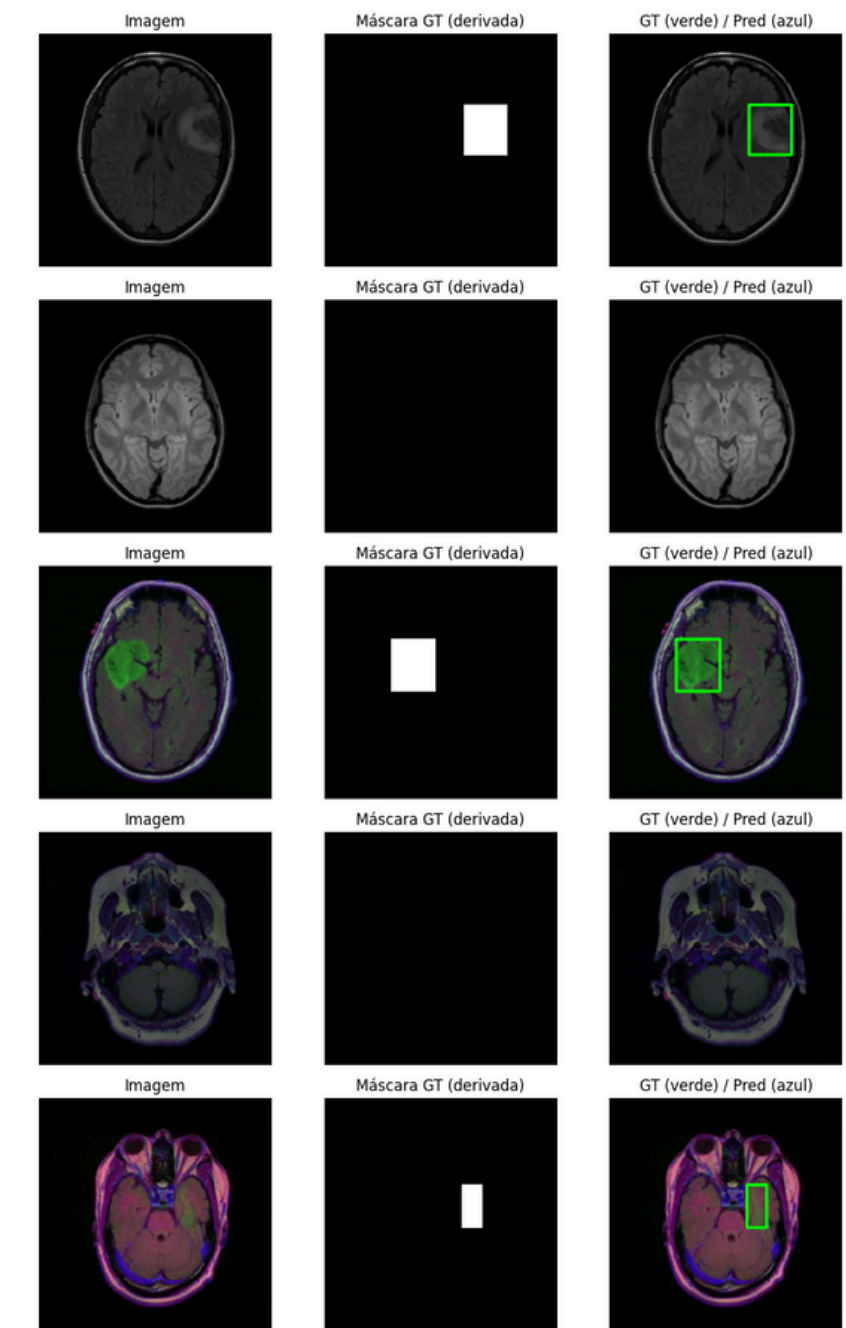
O teste inicial foi treinando segmentação usando DINOv3 ViT-S/16 (PyTorch Hub) (mais leve) no dataset por 50 épocas, batch tamanho 4, total de pares encontrados: 7858 e train=6286 val=785 test=787. Para seguir para detecção transforma as máscaras em rótulos e realiza novo treino por mais 50 épocas. Resultado da detecção: Precision=0.3012 Recall=0.1889.

Em busca da melhoria do resultado o modelo DINOv3 ViT-S/16 foi adaptado descongelando apenas os ultimos blocos e número de epocas 150, adicionando métricas. Novo resultado: mAP@0.50 (1 classe)=0.7883
Precision=0.4592 Recall=0.5486

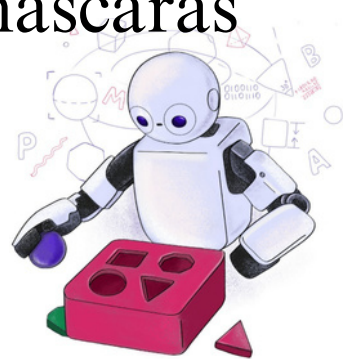


Teste inicial segmentação

Inatel

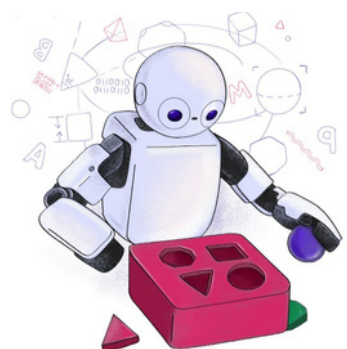


Detecção a partir das máscaras

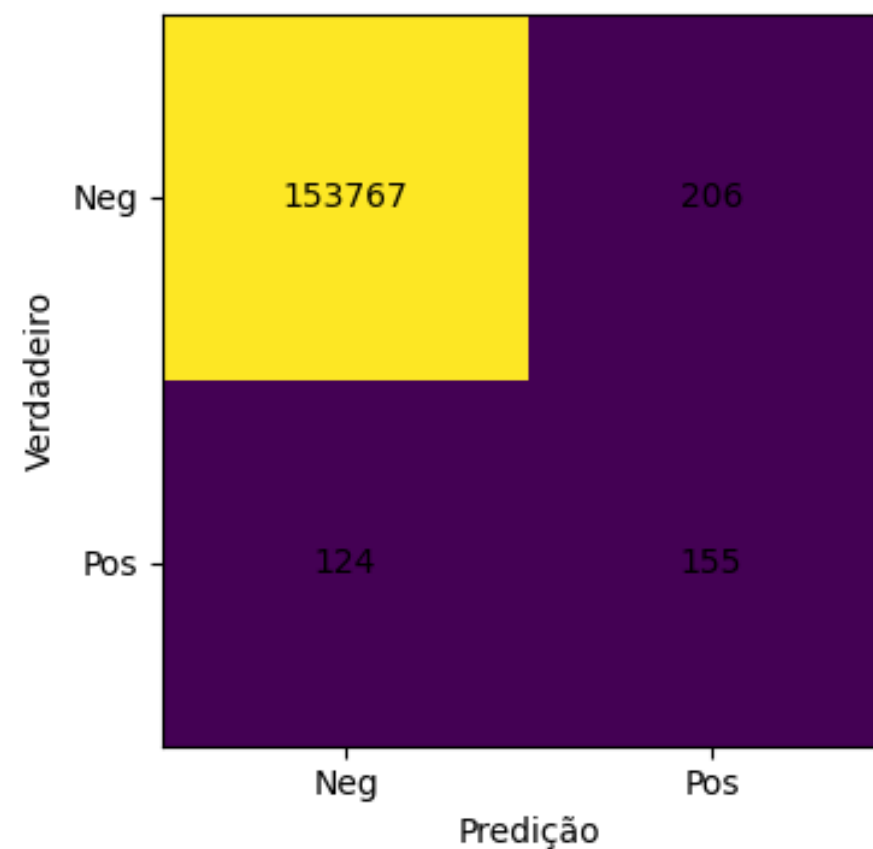


Exemplos de aplicação

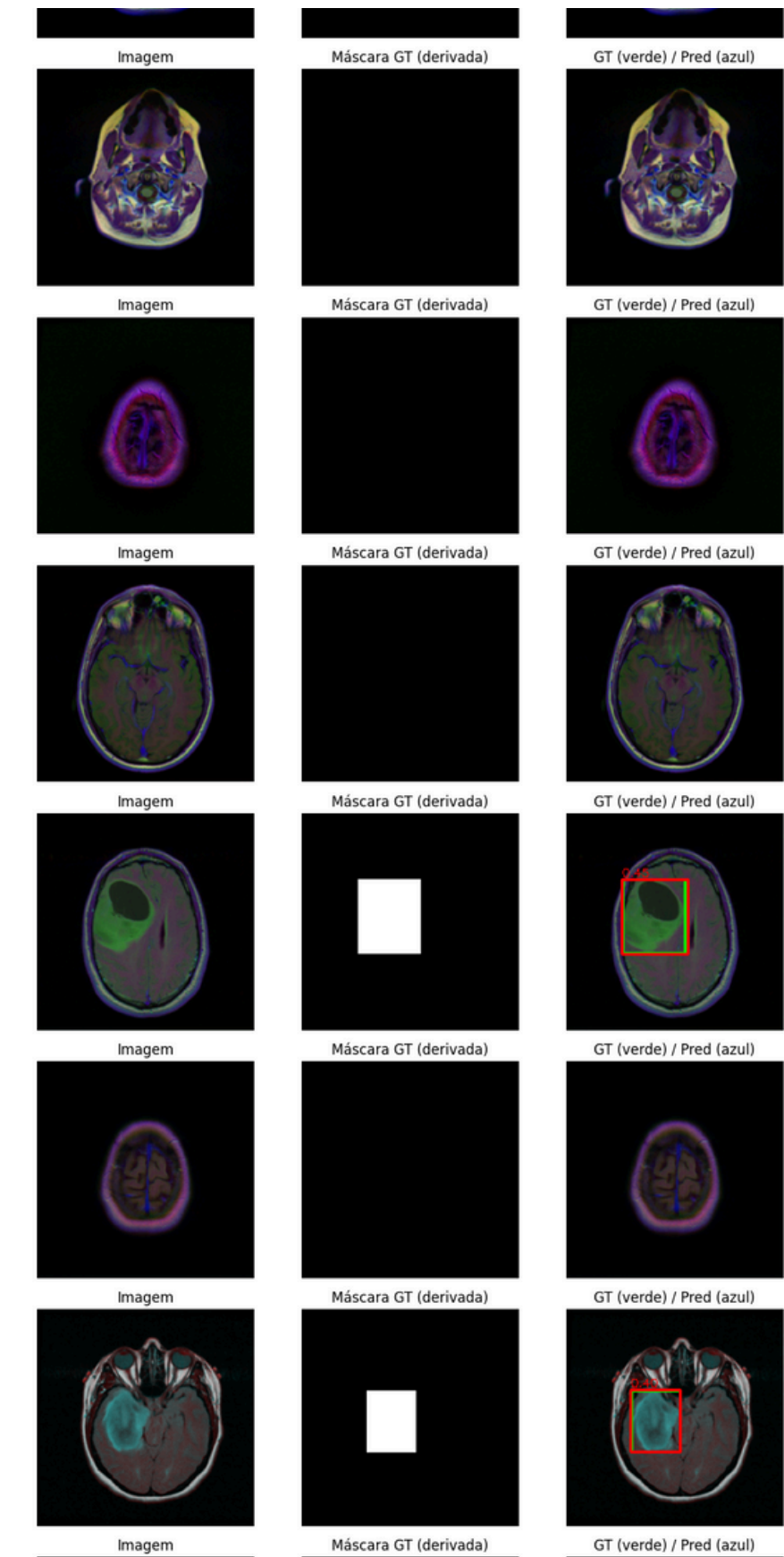
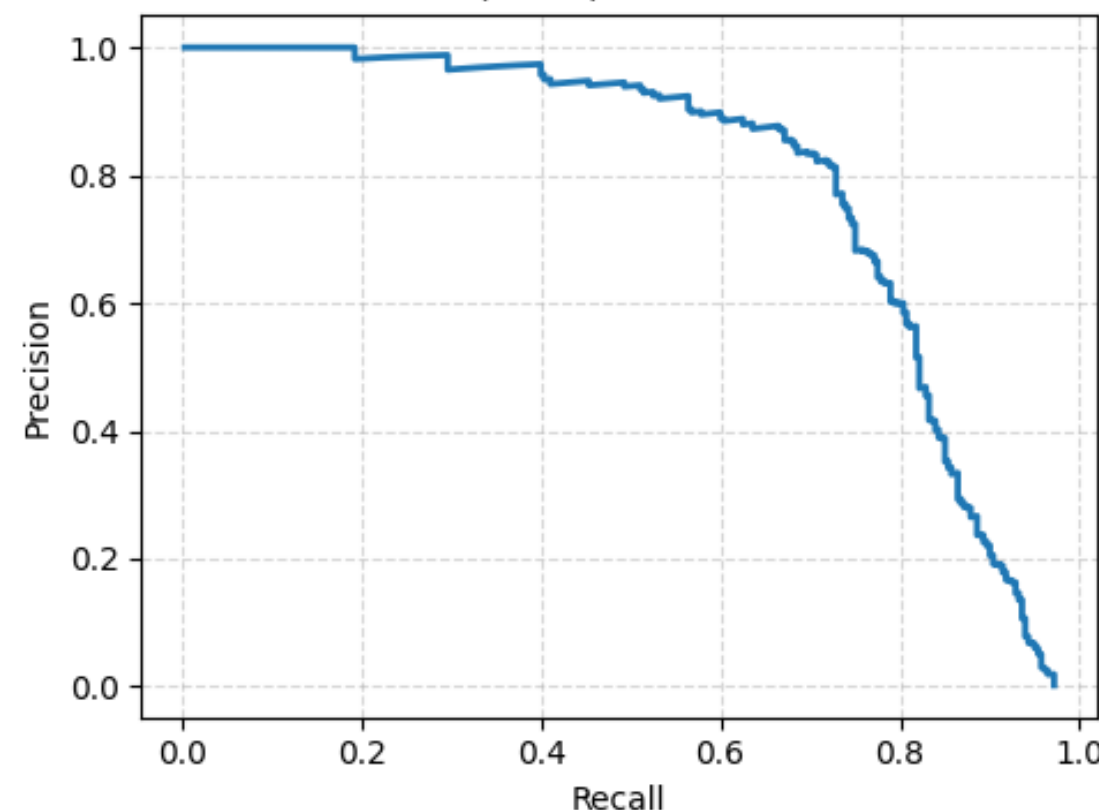
O modelo apresenta boa precisão em baixas taxas de recall, mas perde consistência quando tenta detectar quase todos os objetos. Isso sugere que ele é confiável para detecções conservadoras, mas ainda precisa de ajustes ou mais dados para manter precisão alta em recalls maiores.



Matriz de Confusão (células do heatmap)



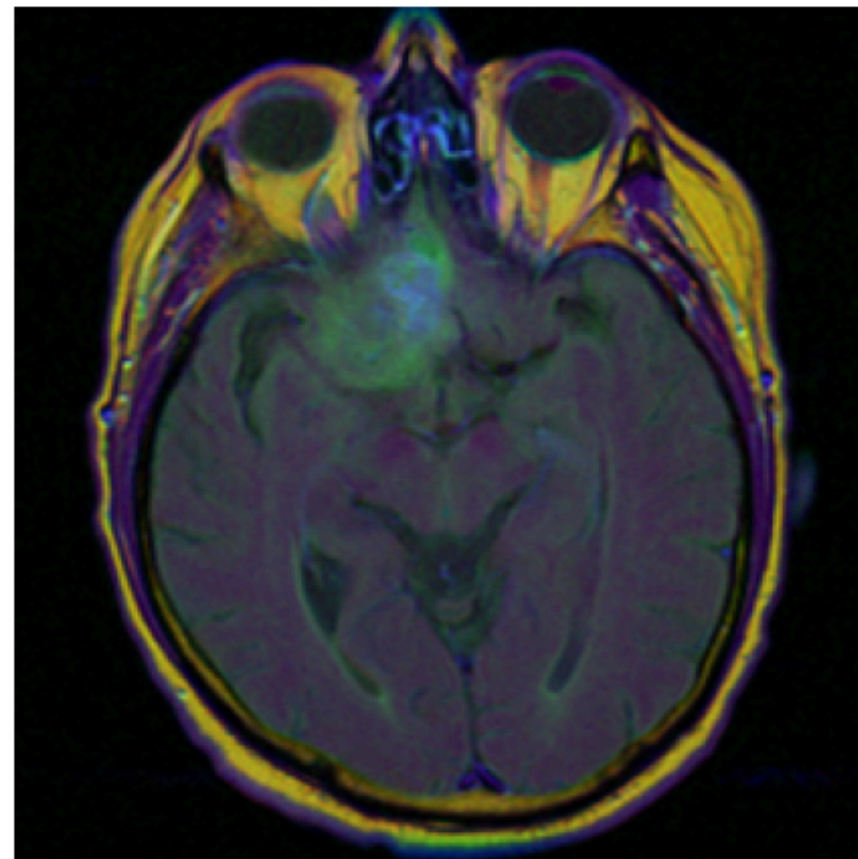
P-R (teste) — $IoU \geq 0.50$



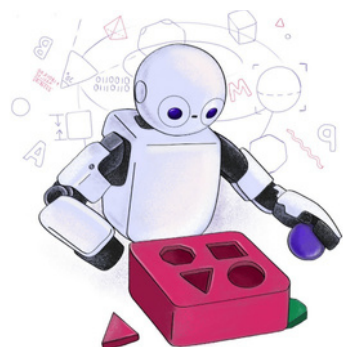
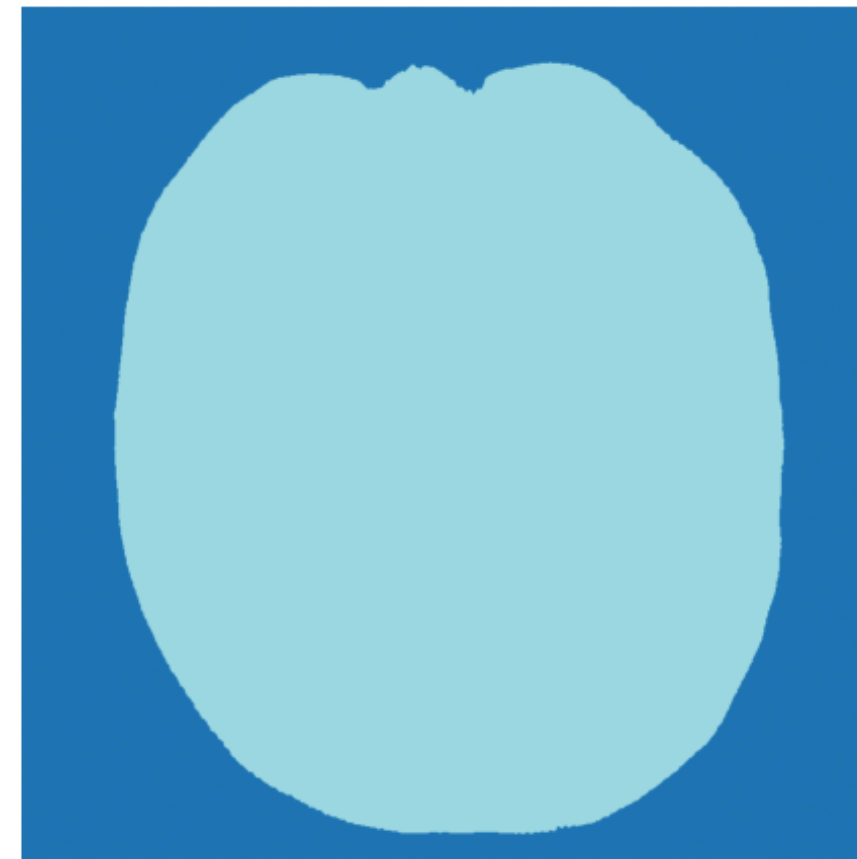
Exemplos de aplicação

Teste de segmentação realizado sem treinamento, usando os pesos do head ADE20K e pesos do backbone ViT-7B/16 e imagem sem máscara. O segmentor tenta atribuir cada pixel a uma das 150 classes do ADE20K (“parede”, “céu”, “pessoa”...), que não têm análogos médicos; o resultado tende a virar um rótulo dominante (um “fundo” qualquer).

Imagem

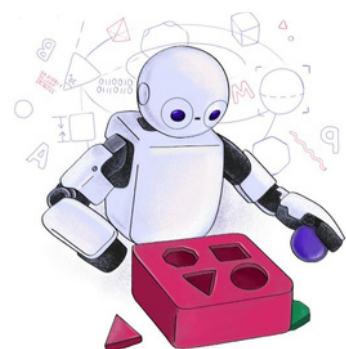
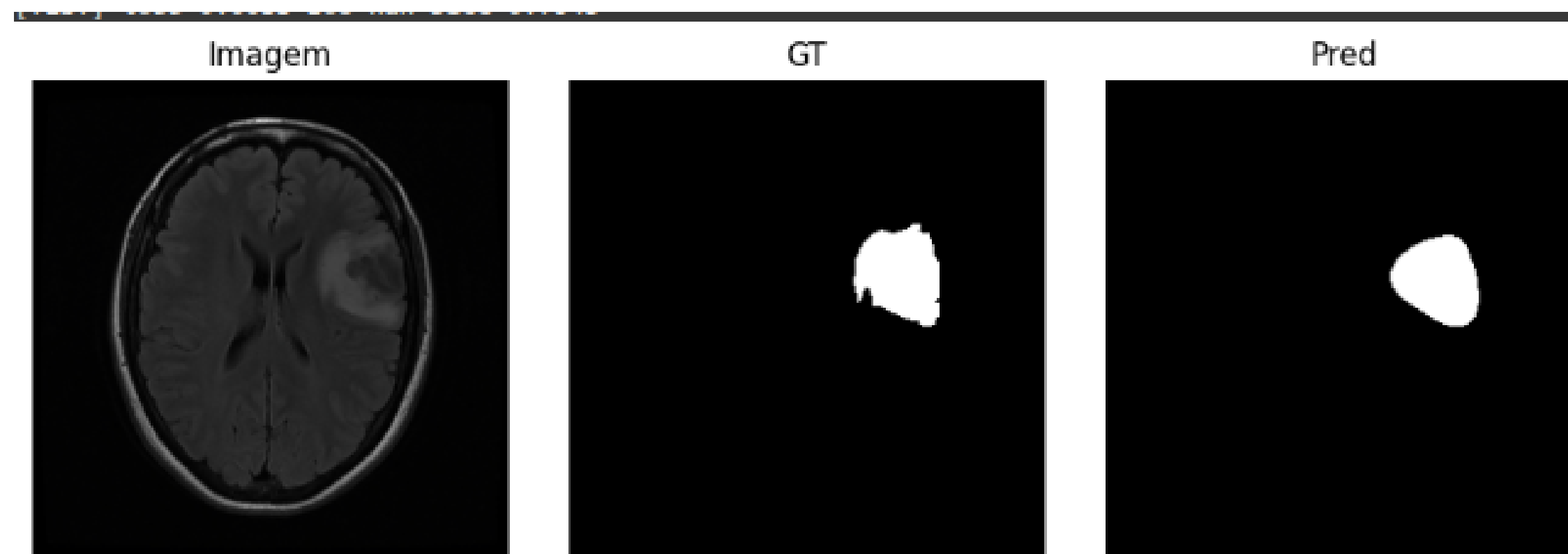


Segmentação (ADE20K)



Exemplos de aplicação

Ultimo teste: O script usa o DINOv3 apenas como backbone congelado (carregado localmente via PyTorch Hub), extraindo tokens de patch e convertendo-os em um feature map 2D [B,C,h,w] (com PATCH=16 e IMG_SIZE=256) para alimentar um decoder leve de segmentação. Os dados vêm do LGG MRI (via kagglehub), com pareamento automático imagem $\leftrightarrow$ *_mask, split 80/10/10 (train/val/test), normalização ImageNet e augs simples (flip, brilho/contraste). Usando ViT-7B/16 (pesado). O decoder (pequeno “UNetzinho”) faz upsampling bilinear progressivo ($\times 16$ no total) com blocos Conv-BN-ReLU e cabeça 1×1 para logit binário. Treina-se só o decoder por 30 épocas, com AdamW e loss mista $0.5\times \text{BCEWithLogits} + 0.5\times \text{Dice}$; as métricas são IoU e Dice/F1 (torchmetrics). A cada época, avalia-se na validação e salva o melhor checkpoint pelo maior Dice de val; no fim, carrega-se esse modelo, mede-se loss/IoU/Dice no teste e gera visualizações Imagem. Resultado: **Melhor val Dice @epoch: (48, 0.788530403917486), loss=0.0633 e Dice=0.7945.**



Obs. Os scripts (notebooks) desenvolvidos para o uso deste trabalho estão disponíveis no github: https://github.com/Rafa-Laureano/TP558---Topicos-avancados-ML/tree/main/Seminario_DINOv3 com nome Dinov3.ipynb onde é possível visualizar os resultados dos treinamentos inclusive.

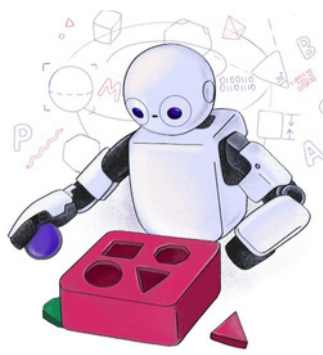
Exemplos de aplicação

Curiosidade: DINOv3 ajudando a restaurar florestas

O World Resources Institute (WRI) usa o DINOv3 para monitorar projetos de restauração florestal e agroflorestal em 27 países africanos. O modelo permite:

- Contar árvores individuais em imagens de satélite e drones, identificando até mudas de 8 meses.
- Medir altura de dossel com >20% mais precisão que o DINOv2, crucial para estimar carbono.
- Unificar dados de múltiplos satélites sem precisar de modelos separados.
- Reduzir custos: ONGs locais conseguem gerar mapas de recuperação florestal com apenas um laptop e \$10 em créditos de nuvem.

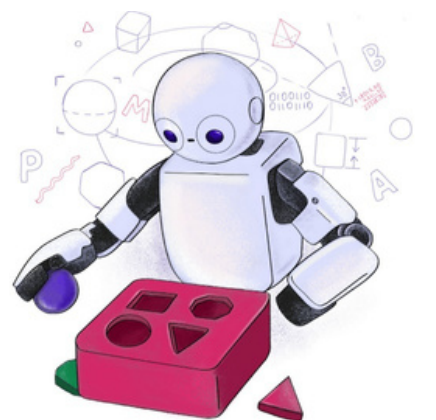
Resultado: mais transparência e confiança em investimentos ambientais, conectando milhões de dólares a projetos de impacto real.



Perguntas?

<https://forms.gle/JT2d7ZjU9rjfyiRY6>

Inatel



Referências

Crie referencias como o exemplo a seguir dos artigos anexos

[1] YM. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging Properties in Self-Supervised Vision Transformers,” arXiv preprint arXiv:2104.14294v2, 2021.

[2] M. Oquab, L. Bottou, I. Laptev, and J. Sivic, “DINOv3: Self-supervised learning for vision at unprecedented scale,” [segundo-artigo.pdf], 2025.

[3] O. de O. Siméoni et al., “DINOv3: Scaling self-supervised learning for vision at unprecedented scale,” arXiv preprint arXiv:2508.10104, 2025.

[4] World Resources Institute and Meta AI, “How DINOv3 is helping World Resources Institute restore nature and measure tree canopy heights from satellite and drone imagery,” Meta AI Blog, Aug. 2025. [Online]. Available: [Lei, M., Li, S., Wu, Y., Hu, H., Zhou, Y., Zheng, X., Ding, G., Du, S., Wu, Z., & Gao, Y. \(2025\). YOLOv13: Real-Time Object Detection with Hypergraph-Enhanced Adaptive Visual Perception. arXiv preprint arXiv:2506.17733.](#)

Obrigado!

Inatel

